Chunking Logic — Preventify Diabetes Educator AI

Version: 2.2

Date: 2026-05-23

Scope: Defines exactly how all 10 parsed sources are split into chunks, what metadata each chunk carries, and how the chunker handles tables, safety-critical content, and sources without clean boundaries.

This document supersedes v1.0 and the chunking notes in `base_model_spec.md` Section 3a.

Implement in `ingestion/chunkers/`.

Design principle

This bot serves normal patients. They never see metadata — they see only the answer Claude generates.

Metadata has one job: **pre-filter chunks before semantic search** so the right pool of chunks reaches the reranker. Think of it as a SQL WHERE clause, not a display field.

Every field kept below earns its place by enabling a concrete filter. Fields that exist only for citations, audit trails, or developer inspection are dropped.

Chunker type assignment

Source	Chunker	Reason
RSSDI_2022	recommendation	Split at rag_metadata comment boundaries
ICMR_STW_2024	recommendation	Split at rag_metadata comment boundaries
ADA_2026	recommendation	Split at rag_metadata comment boundaries across 15 merged sections
KDIGO_2022_DM_CKD	page_window	No annotation passes yet; raw page text chunked as 2-page windows
ESC_2023_CVD_DM	recommendation	Split at rag_metadata comment boundaries
Anoop_Misra	narrative	Section annotations only; no recommendation-boundary markers
WHO_HEARTS	narrative	Section annotations; no recommendation markers
Telemedicine_Guideline	narrative	Section annotations; compliance namespace
IDF_DAR	page_window	Only page markers; annotated safety/meal/risk lines extracted first

ICMR_NIN	food_table	Food group tables with Kerala sub-tables

Chunk text format — applies to all types

Every chunk's `text` field starts with a context header. The format is conditional on whether a `section_ref` (numbered section ID) is present alongside `section_title`:

```
[{SOURCE} {YEAR} — {SECTION_REF}: {SECTION_TITLE}]   ← when both present
[{SOURCE} {YEAR} — {SECTION_TITLE}]                   ← section_title only
[{SOURCE} {YEAR}]                                     ← no section info
```

Most sources produce the second form. Sources with explicit numbered section IDs (e.g. "S3.2") in their headings produce the first form.

Without this header, a chunk like "HbA1c target < 7.0%" has no indication it is for non-elderly adults and from an India-specific source. The embedding of header + body together produces a richer vector than body alone.

Table chunks use the same header plus a table caption line:

```
[{SOURCE} {YEAR} — {SECTION_TITLE}]
Table: {caption or first heading above the table}

| col1 | col2 | ... |
| --- | --- | ... |
| row  | ...  | ... |
```

---

1. Recommendation chunker

Used for: RSSDI_2022, ICMR_STW_2024, ADA_2026, ESC_2023_CVD_DM

Split logic

The parsed markdown files have `<!-- rag_metadata ... -->` comments embedded at recommendation and section boundaries. These are the split points — not line counts, not token counts.

```
Step 1 — scan the file sequentially.
Step 2 — when a <!-- rag_metadata ... --> comment is encountered:
          • finalize the current chunk (everything since the previous metadata comment)
          • start a new chunk
          • the metadata comment fields become this chunk's metadata
Step 3 — find the nearest ## or ### heading above this metadata comment; that is section_title.
Step 4 — prepend the context header to the chunk body text.
Step 5 — apply the token ceiling check (see below).
```

Token ceiling: 512 tokens

Approximate token count = `len(text) // 4`.

Situation	Action
full chunk ≤ 512 tokens	emit as single chunk
full chunk > 512 tokens, safety_critical=true	emit as-is — NEVER split, no ceiling applies
full chunk > 512 tokens, table content	token-aware row batching — see table rules below
full chunk > 512 tokens, recommendation/narrative text	paragraph → sentence → hard-split cascade (see below)

Splitting cascade — `split_text_with_ceiling()` in `ingestion/chunkers/base.py`:

The effective body budget is `512 - header_tokens - 4` (the 4-token buffer absorbs rounding in the `len//4` estimate). Three passes run in order until every fragment fits:

```
Pass 1 — paragraph split (split at \n\n boundaries)
  → accumulate paragraphs greedily until adding the next would exceed body_max
  → emit current window, start new window

Pass 2 — sentence split (split at ". [A-Z]" / "! [A-Z]" / "? [A-Z]")
  → applied to any paragraph that still exceeds body_max after Pass 1
  → each sentence is itself checked; if a single sentence > body_max, falls to Pass 3

Pass 3 — hard split (word-boundary cut, last resort)
  → applied when no sentence boundaries exist (e.g. RSSDI pdfplumber concatenated text)
  → splits at last space before max_chars = body_max × 4
  → if no space found (pure concatenated run), cuts at exact character position
```

Every fragment repeats the context header so it is fully self-contained. The context header is NOT included in the body budget calculation — it is always prepended on top.

Accurate token measurement — assemble-and-measure:

All chunkers compare the **assembled candidate text** against the ceiling, not accumulated per-fragment estimates. This avoids the integer floor-division rounding error that arises when summing `len(x) // 4` across fragments: the sum of parts can be less than the estimate of the joined whole by up to one token per separator character. Assembling the full text first gives a single correct estimate.

```
# Wrong (accumulated estimates — can undercount by N tokens for N separators):
current_tok += token_estimate(row + "\n")
if current_tok > _MAX_TOKENS: flush()

# Correct (assemble-and-measure):
candidate = f"{header}\n\n{table_header}\n" + "\n".join(current_batch + [row])
if token_estimate(candidate) > _MAX_TOKENS: flush()
```

---

2. Narrative chunker

Used for: Anoop_Misra, WHO_HEARTS, Telemedicine_Guidelines

Split logic

```
Step 1 — split at ## and ### heading boundaries; each heading opens a new section.
Step 2 — for each section:
          • count tokens
          • if ≤ 512: entire section = 1 chunk
          • if > 512: apply _split_with_overlap():
              a. Compute body budgets (see overlap section below)
              b. Pre-expand any paragraph > body_max_rest via sentence → hard-split cascade
              c. Accumulate expanded paragraphs greedily; compare assembled candidate text
                 against _MAX_TOKENS (assemble-and-measure)
              d. Emit window; carry 50-token overlap tail into next window
              e. Each fragment gets the context header repeated
          • if section is < 50 tokens: skip (too small to embed meaningfully)
Step 3 — metadata comes from the rag_metadata comment attached to that section heading.
          If no metadata comment on a heading, inherit from the nearest one above.
```

Overlap implementation

The narrative chunker carries a 50-token overlap tail between fragments to prevent splits from severing mid-sentence clinical claims that bridge paragraph boundaries. Understanding the two-budget design is important for correctness:

Two body budgets, not one:

```
body_max_first = max(64, 512 - header_cost - 4)
    ↑ used for the FIRST fragment (no overlap tail prepended, so full budget available)

body_max_rest  = max(64, 512 - header_cost - 50 - 4)
    ↑ used for ALL SUBSEQUENT fragments (overlap tail consumes ~50 tokens)
```

The original single-budget design wasted 50 tokens on every first fragment. The corrected design uses the larger budget for the first fragment and the tighter budget for all subsequent ones.

Pre-expansion uses `body_max_rest`:

Before the greedy accumulation loop, any paragraph that exceeds `body_max_rest` is expanded via sentence → hard-split cascade. This uses the tightest budget, not the first-chunk budget. Reason: an expanded paragraph that fits within `body_max_first` (490 tokens) may later be placed in a rest-chunk position where the overlap tail is also prepended — it would then exceed 512. Expanding to `body_max_rest` (440 tokens) ensures every expanded fragment is safe regardless of which position it lands in.

Overlap tail extraction — `_last_n_tokens(text, 50)` in `narrative.py`:

```
def _last_n_tokens(text: str, n: int) -> str:
    max_chars = n * 4          # 50 tokens × 4 chars/token = 200 chars
    if len(text) <= max_chars:
        return text
    start_pos = max(0, len(text) - max_chars)
    # Walk forward to next word boundary — avoids cutting mid-word
    while start_pos < len(text) and text[start_pos] not in (" ", "\n"):
        start_pos += 1
    return text[start_pos:].lstrip()
```

Uses character-budget (`n × 4`) to approximate token count, then aligns to the next word boundary. Because it walks forward (increasing start_pos), the returned tail is always ≤ 200 chars ≤ 50 tokens. Earlier version counted words (`" ".join(words[-50:])`) which returned ~65 actual tokens for typical clinical prose — a systematic 30% overrun.

Overlap prepend:

```
# Only prepend overlap on rest-chunks (chunks[0] has no overlap)
body_text = overlap_tail + "\n\n" + body_text  # if overlap_tail and chunks
```

The overlap tail appears at the top of the body (below the context header), not the bottom. This means the reranker sees the bridging text at the start of the fragment where it provides context for the continuation.

Assemble-and-measure for accumulation:

The greedy loop assembles the full candidate chunk text (header + optional overlap tail + candidate paragraphs) and measures token count on the assembled text. This avoids the floor-division rounding error from summing per-paragraph estimates.

---

3. Page-window chunker

Used for: IDF_DAR, KDIGO_2022_DM_CKD

The IDF-DAR file is 333 pages of raw pdfplumber text with `<!-- page N -->` markers and inline annotation comments. There are no section-level boundaries to split on.

Step 1 — extract priority chunks first (before page windowing)

Safety-redline chunks (`safety_redline=true`):

- Include the annotated line + 3 lines of surrounding context
- BG thresholds for breaking the Ramadan fast: < 70 mg/dL, > 300 mg/dL
- metadata: safety_critical=true, condition_trigger=ramadan
- NEVER split. NEVER include in a page window. Always standalone.

Meal-timing chunks (`meal_context=suhoor` or `meal_context=iftar`):

- Include the annotated line + next 4 lines
- metadata: meal_context=suhoor|iftar, condition_trigger=ramadan
- One chunk per annotated block. Keep atomic.

Risk-stratification chunks (`chunk_note=keep_atomic_large_window`):

- Include the annotated block until the next <!-- page N --> marker
- IDF-DAR risk category scoring rows (Very High / High / Moderate / Low)
- metadata: condition_trigger=ramadan
- Keep atomic even if > 512 tokens.

Step 2 — slide a 2-page window over remaining text

After removing lines consumed in Step 1:

- Window = 2 consecutive pages
- Overlap = 1 page (page N is shared between window [N-1, N] and window [N, N+1])
- Token ceiling enforced: every window passes through _emit_window_chunks() which applies the full paragraph → sentence → hard-split cascade if the 2-page window exceeds 512 tokens

Design note: IDF-DAR averages ~250 tokens/page so 2 pages ≈ 500 tokens — usually fits in one chunk. KDIGO is denser (tested at up to 4,387 tokens before ceiling enforcement); the cascade splits these into sub-512 fragments automatically.

---

4. Food table chunker

Used for: ICMR_NIN only

The ICMR-NIN parsed file has two levels of tables per group:

- ## Group Name — full group table (all foods in that IFCT group)
- ### Group Name — Kerala Relevant Foods — sub-table of Kerala-relevant rows only

Emit **two types of chunks** per group that has Kerala foods:

Type A — group-level chunks

"Which Kerala fish have the highest protein?" — needs the full group table.

- One chunk per ## Group Name section
- If group table ≤ 512 tokens: entire table = 1 chunk
- If group table > 512 tokens: token-aware batching — rows are accumulated greedily

until adding the next row would push the chunk over 512 tokens; a hard ceiling of 30

rows per batch also applies; column header row repeated at the top of every batch;

context header prepended to every batch

Type B — individual Kerala food chunks

"How many carbs in karimeen?" — needs a single-row chunk.

- One chunk per row in ### Group Name — Kerala Relevant Foods sub-tables
- Format: context header + column header row + single food row
- metadata: kerala_food=true

```
[ICMR-NIN 2017 — Marine Fish — Kerala Relevant Foods]

| Food Code | Food Name | Carb (g) | Protein (g) | Fat (g) | Fiber (g) | Energy (kJ) |
| --- | --- | --- | --- | --- | --- | --- |
| P026 | Karimeen (Etroplus suratensis) | 386.0 | 78.66 | 0.97 | | |
```

Type A handles comparison queries; Type B handles specific food lookups. Both enter the same pgvector collection.

Table rules (applies to all chunkers)

Condition	Rule
Table ≤ 512 tokens	Single chunk
Table > 512 tokens,	Single chunk regardless of size — zero-loss
Table > 512 tokens,	Token-aware batching: accumulate rows until next row would exceed 512 tokens; hard cap of 30
Table row spans	Treat the full row as atomic — never split inside a row
multiple lines	

Evidence grade → grade_priority

Five different grading schemes across 10 sources. The only field that matters for retrieval is `grade_priority` (integer 1–5). It is a hardcoded lookup — no ML, no inference.

`grade_priority` is the pre-filter for safety-critical queries:

```
patient asks about hypoglycemia → filter: grade_priority <= 2
→ only Grade A/B evidence chunks enter semantic search
→ expert-opinion chunks are excluded
```

```
GRADE_PRIORITY = {
    # ADA / RSSDI
    "A": 1, "B": 2, "C": 3, "E": 4,

    # KDIGO — two-axis grading:
    #   first digit = recommendation strength (1 = strong/"We recommend",
    #                                          2 = weak/"We suggest")
    #   letter      = evidence quality (A = high, B = moderate, C = low, D = very low)
    # Strong recs always rank higher than weak recs at the same evidence quality.
    # E.g. 1B (strong/moderate, priority 2) > 2A (weak/high, priority 3).
    "1A": 1,   # strong rec, high evidence
    "1B": 2,   # strong rec, moderate evidence
    "1C": 3,   # strong rec, low evidence
    "1D": 4,   # strong rec, very low evidence
    "2A": 3,   # weak rec, high evidence   ← priority 3, not 2
    "2B": 3,   # weak rec, moderate evidence
    "2C": 4,   # weak rec, low evidence
    "2D": 5,   # weak rec, very low evidence

    # ESC — merge evidence_class + evidence_level first (e.g. "I-A", "IIb-C")
    # Class I=1, IIa=2, IIb=3 | Level A/B=+0, C=+1 → cap at 5
    # Class III (harm/no benefit) → priority 5 + safety_critical=True (see below)
}

CONSENSUS_PRIORITY = 5  # no formal grade (WHO HEARTS, Anoop Misra)
```

KDIGO note — why 2A = 3, not 2:

KDIGO 1B and 2A previously both mapped to priority 2. The first digit encodes recommendation strength (1 = "We recommend" / strong; 2 = "We suggest" / weak) — a clinically meaningful distinction. A strong recommendation with moderate evidence (1B) outranks a weak recommendation with high evidence (2A). Both axes are now encoded: strong recs always rank one level higher than weak recs at the same evidence quality.

ESC Class-III special rule:

ESC `Class-III` means the intervention is **not recommended or potentially harmful** — it is not a grading tier but a contraindication signal. These chunks always get:

- grade_priority = 5
- safety_critical = True (set by the recommendation chunker, not just the extractor annotation)

The bot retrieves Class-III chunks precisely *because* they are harmful — the RAG response must know what NOT to advise. They are flagged so the response generation layer can handle them as "do not recommend" signals, not as evidence to cite in favour of the intervention.

ESC case normalization:

`normalize_esc_grade()` normalises class and level to uppercase internally before lookup, so `"IIa"`, `"IIA"`, and `"iia"` all resolve correctly. Previously the dict used mixed-case keys (`"IIa"`, `"IIb"`) which caused silent priority-5 fallback when callers passed uppercase values (as the table scanner does via `.upper()`).

Unrecognised grade strings:

Any grade string not in the lookup table gets `grade_priority = 5` and a `WARNING` log line (via Python `logging`). This surfaces extractor annotation gaps during corpus development without stopping the pipeline.

For ESC chunks: the extractor produces `evidence_class` (I/IIa/IIb/III) and `evidence_level` (A/B/C) as two fields. The chunker resolves these to a single `grade_priority` integer and does NOT store either raw field.

Chunk metadata schema

14 fields. No more.

```
{
  "chunk_id":          "a3f2b89c1d4e5f67",
  "source":            "RSSDI_2022",
  "year":              2022,
  "section_title":     "Glycemic Targets",
  "text":              "[RSSDI 2022 — Glycemic Targets]\n\nHbA1c target...",
  "retrieval_tier":    "core",
  "condition_trigger": null,
  "india_specific":    true,
  "kerala_food":       false,
  "safety_critical":   false,
  "grade_priority":    1,
  "meal_context":      null,
  "text_hash":         "a3f2b89c1d4e5f67a3f2b89c1d4e5f67",
  "token_estimate":    87
}
```

Field reference

Field	Type	What it does
chunk_id	string	Deterministic hash — stable across re-runs; used for targeted re-ingestion
source	string	Which guideline this came from; goes into the context header
year	int	Publication year; goes into the context header
section_title	string	Section name from the original document; goes into the context header
text	string	Full chunk text including context header — this is what gets embedded and what Claude
retrieval_tier	string	core = always retrieved \

condition_trigge	string\	null
india_specific	bool	true for RSSDI/ICMR sources — used to prefer Indian guidelines over international for same
kerala_food	bool	true only on ICMR-NIN Type B individual Kerala food row chunks
safety_critical	bool	true = never split, always retrieved on safety queries, zero-loss
grade_priority	int	1–5 — pre-filter for safety queries; 1=strongest evidence, 5=consensus/ungraded
meal_context	string\	null
text_hash	string	SHA256 of body text — dedup at upsert; indexed in pgvector
token_estimate	int	len(text) // 4 — cost tracking and token budget awareness

Fields intentionally dropped

Dropped field	Why
section_ref	Not filtered on; used only in chunk_id — replaced with section_title hash
evidence_grade	Raw grade string; grade_priority already does the retrieval job
evidence_schema	Which grading scale — internal, never queried
content_type	Not filtered on in any patient query
topic_tags	pgvector filters are binary — can't boost by tag
fragment	Internal chunker bookkeeping
chunk_note	Used during chunking only; irrelevant after
page_range	IDF-DAR page numbers; patient never benefits
duplicate_of	Used at upsert time only; not stored permanently
char_count	Redundant with token_estimate

Chunk ID generation

Deterministic. Stable across re-runs as long as source + section + body don't change.

```
import hashlib

def make_chunk_id(source: str, section_title: str | None, text: str) -> str:
    ref = section_title or "nosec"
    fingerprint = f"{source}|{ref}|{text[:200]}"
    return hashlib.sha256(fingerprint.encode()).hexdigest()[:16]
```

When RSSDI 2023 ships, chunks that hash-match existing store entries can be skipped — only changed or new chunks need re-embedding.

Deduplication at upsert time

Several guidelines repeat identical recommendations across sections. Duplicates waste reranker budget.

```
1. Hash the chunk body text (SHA256, excluding the context header) → text_hash
2. Check if text_hash already exists in pgvector payload index
3. If it exists:
   - Keep the one with lower grade_priority (stronger evidence)
   - If grade_priority is equal, keep the one with higher year (more recent guideline)
   - Do NOT upsert the weaker copy
```

`text_hash` must be a payload index in pgvector for this lookup to be fast.

How retrieval uses metadata

```
Normal query — "Can I eat rice?":
  filter: retrieval_tier = "core"
  → semantic search on ~800 chunks → reranker → top 5 → Claude

Patient has CKD — "How much protein?":
  filter: retrieval_tier IN ("core", "triggered")
          AND condition_trigger IN (null, "ckd")
  → KDIGO chunks included; override Tier 1 on kidney queries

Ramadan patient — "Can I take my tablet during roza?":
  filter: retrieval_tier IN ("core", "triggered")
          AND condition_trigger IN (null, "ramadan")
  → suhoor/iftar meal_context chunks prioritised

Any safety question — hypoglycemia, BG thresholds:
  safety_critical = true chunks always retrieved first
  THEN grade_priority <= 2 for the remaining pool

Kerala food lookup — "How many carbs in karimeen?":
  filter: kerala_food = true
  → single-row ICMR-NIN chunk comes straight to top
```

Per-source notes

RSSDI_2022

- Inline grades (A), (B), (C), (E) already annotated by extractor — read directly.
- Sections without a grade: grade_priority = 5.

ADA_2026

- 15 section PDFs merged into one file. Section separators are

`<!-- source: ADA_2026 | file: ADA_2026_Sxx.pdf | ... -->` comments.

Use `Sxx` as the section seed for `section_title` before the finer rag_metadata splits.

- Grade format in annotation: grade_A, grade_B — strip the grade_ prefix before lookup.

KDIGO_2022_DM_CKD

- Currently uses page_window chunker — no _annotate_* passes exist yet in the extractor.
- Adding eGFR-threshold and recommendation-block annotation passes is deferred (see CHUNKING_DISCUSSION.md "What is NOT done yet" item 4). When that work is done, switch to recommendation chunker and mark eGFR-threshold chunks safety_critical=true.

ESC_2023_CVD_DM

- Extractor produces evidence_class and evidence_level as two fields.

Resolve to `grade_priority` integer in the chunker; do not store either raw field.

ICMR_STW_2024

- Algorithm step lines have chunk_note=keep_atomic_large_window — keep the full step together.

A split treatment step is clinically dangerous.

IDF_DAR

- Extractor reports: 13 safety-redline annotations, 17 meal-timing annotations, 97 risk annotations.
- Extract all of these as priority standalone chunks before sliding the page window.
- Actual output: 1,103 chunks after ceiling enforcement splits page-window blocks that exceeded 512 tokens.

WHO_HEARTS

- Step-up protocol chunks are atomic — the 6-step antihypertensive ladder must never be split.

A fragment starting at Step 3 with no Step 1/2 context is clinically dangerous.

- BP threshold chunks (safety_critical=true): zero-loss.

ICMR_NIN

- Use ### ... — Kerala Relevant Foods headings to identify Type B individual chunks.
- Food names in many rows are garbled (PDF parsing noise). Do NOT clean in chunker — chunk as-is.

The Latin species name embedding (e.g. "Etroplus suratensis") still matches relevant queries.

Output format

Each chunker writes to `data/chunks/{SOURCE}.jsonl` — one JSON object per line.

```
data/chunks/RSSDI_2022.jsonl
data/chunks/ADA_2026.jsonl
data/chunks/ICMR_STW_2024.jsonl
data/chunks/ICMR_NIN.jsonl
data/chunks/KDIGO_2022_DM_CKD.jsonl
data/chunks/ESC_2023_CVD_DM.jsonl
data/chunks/Anoop_Misra.jsonl
data/chunks/WHO_HEARTS.jsonl
data/chunks/IDF_DAR.jsonl
data/chunks/Telemedicine_Guidelines_2020.jsonl
```

What the chunker does NOT do

- Does not clean garbled food names in ICMR-NIN rows — separate data quality task
- Does not translate to Malayalam — embedding is English-only at this stage
- Does not validate clinical accuracy — that is B2/B3 clinical sign-off
- Does not write to pgvector — that is the embedder + upsert step